MODULE 1

INTRODUCTION TO OPERATINGSYSTEMS, STRUCTURES

OPERATING SYSTEM

An Operating System is a Program that manages the computer hardware and acts as an intermediary between the user of the computer & the computer hardware.

✓ The goal of OS:

- To execute programs.
- To make solving user problems easier.
- To make the computer convenient to use.

What Operating systems do

✓ A computer system can be roughly divided into four components:

- The Hardware,
- The Operating System,
- The Application Program,
- The Users.

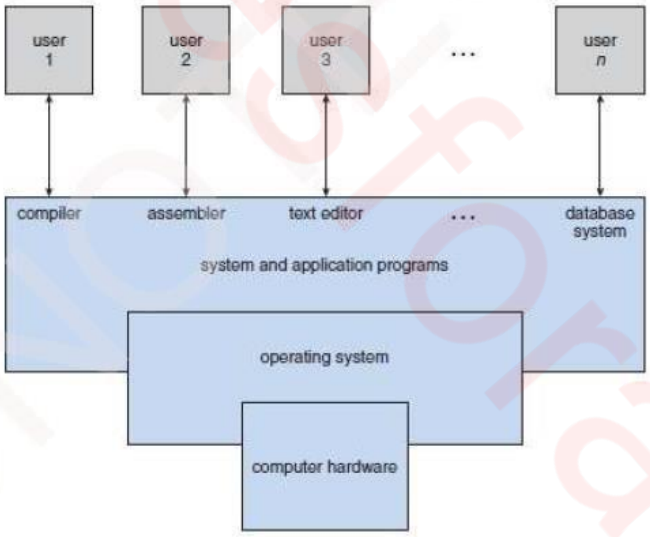


FIG: ABSTRACT VIEW OF COMPONENTS OF A COMPUTER SYSTEM

✓ The OS must support the following tasks:

a. Provide the facility to create, modification of programs & data files using on **Editors**.

b. Access to **Compilers** for translating the user program from high level language to machine language.

c. Provide a **Loader** program to move the compiled program code to computers memory for execution.

d. Provides **Routines** that handle the details of I/O programming.

✓ The Role of operating systems can be viewed from two viewpoints:

- The user and
- The system.

User Views:

The user view of the computer depends on the interface used.

- Some users may **use PC's.** In this **the system is designed so that only one user can utilize the resources and mostly for ease of use where the attention is mainly on performances** and not on the resource utilization.

- Some **users may use a terminal connected to a mainframe or minicomputers**. Other users may access the same computer through other terminals. **These users may share resources and exchange information. In this case the OS is designed to maximize resource utilization**-so that all available CPU time, memory & I/O are used efficiently.

- **Other users may connect to the networks of other work station and servers. In this case OS is designed to compromise between individual visibility & resource utilization.**

System Views:

✓ The OS acts as a manager of resources.

- The OS must decide how to allocate resources to programs and the users so that it can operate the computer system efficiently and fairly.

- Resources can be either
  1. CPU Time,
  2. memory space,
  3. file storage space,
  4. I/O devices and so on.

✓ An OS is a control program

The OS is needed to control

- Operation of I/O devices and
- Execution of user program to prevent errors.

vtunotesforall.in

Computer-System Organization

✓ Computer-System Operation:

- A modern general-purpose computer system consists of

1. one or more CPUs and
2. a number of device controllers connected through a common bus that provides access to shared memory (Figure 1.2).

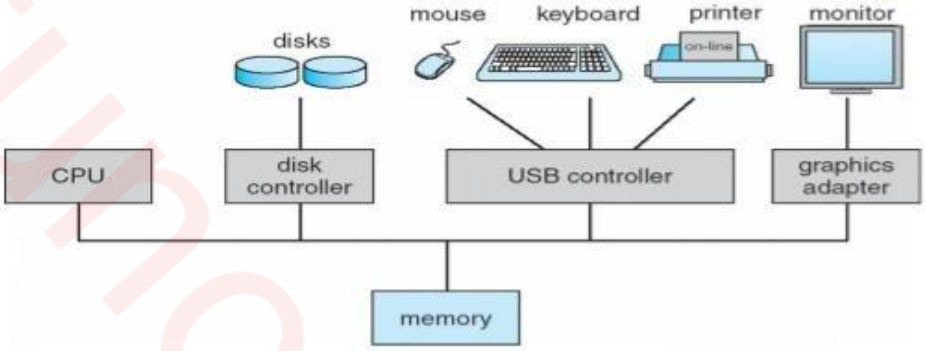


Figure 1.2 A modern computer system.

- Each device controller is in charge of a specific type of device (for example, disk drives, audio devices, and video displays).

- The CPU and the device controllers can execute concurrently, competing for memory cycles.

✓ For a computer to start running—for instance, when it is powered up or rebooted—

- The initial program, or **bootstrap program,** is executed when the system is powered on.
  Bootstrap program is stored in read-only memory (ROM) or electrically erasable programmable read-only memory (EEPROM).

- The bootstrap program must load the operating system.

- Then the OS starts executing the first process such as init and waits for some event to occur.

- When the CPU is interrupted, it stops what it is doing and immediately transfers execution to interrupt service routine (ISR). The ISR executes; on completion, the CPU resumes the interrupted computation.

  A time line of this operation is shown in Figure 1.3. The interrupt must transfer control to the appropriate interrupt service routine.

vtunotesforall.in

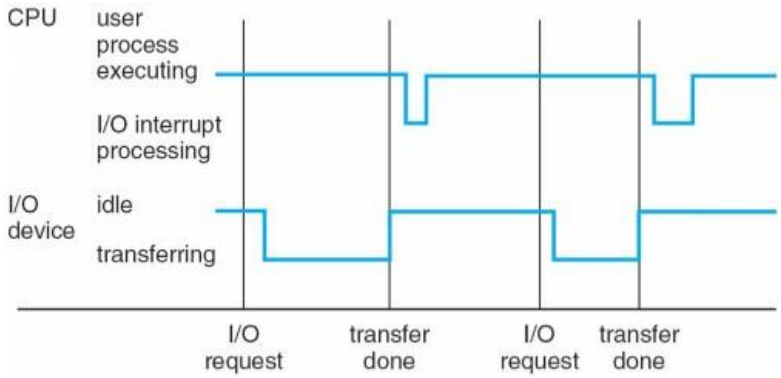


Figure 1.3 Interrupt time line for a single process doing output.

Storage Structure

- Computer programs must be in main to be executed. Main memory is the only large storage area that the processor can access directly.
- Interaction is achieved through a sequence of load or store instructions to specific memory addresses. The load instruction **moves a word from main memory to an internal register within the CPU**, whereas the **store instruction moves the content of a register to main memory.**
- We want the programs and data to reside in main memory permanently. This arrangement usually is not possible for the following two reasons:
    1. Main memory is usually too small to store all needed programs and data permanently.
    2. Main memory is a volatile storage device that loses its contents when power is turned off or otherwise lost.
- Most computer systems provide secondary storage as an extension of main memory. The main requirement for secondary storage is that it be ***able to hold large quantities of data permanently.***
- The most common secondary-storage device is a ***magnetic disk, which provides storage for both programs and data.***
- The wide variety of storage systems in a computer system can be organized in a hierarchy (Figure 1.4) according to speed and cost.
- The higher levels are expensive, but they are fast. As we move down the hierarchy, the cost per bit generally decreases, whereas the access time generally increases.

vtuquestionpaper.in

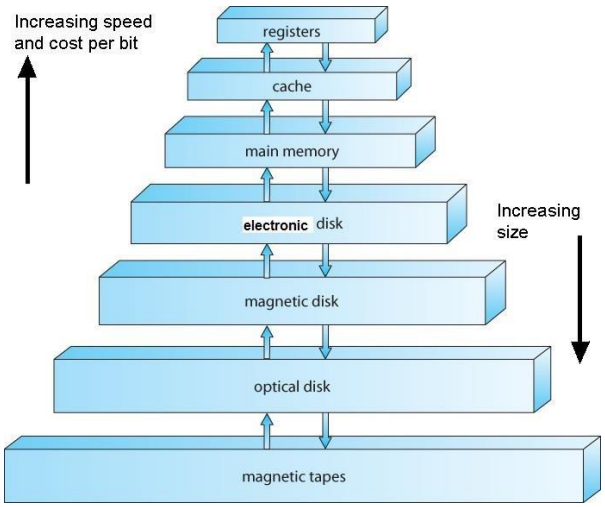


Figure 1.4 Storage-devices Hierarchy

I/O Structure

- ✓ A computer system consists of CPUs and multiple device controllers that are connected through a common bus. Each device controller is in charge of a specific type of device.
- ✓ A device controller maintains some local buffer storage and a set of special-purpose registers. The device controller is responsible for moving the data between the peripheral devices that it controls and its local buffer storage.
- ✓ Typically, operating systems have a **device driver** for each device controller.
- ✓ This device driver understands the device controller and presents a uniform interface to the device to the rest of the operating system.
- ✓ To start an I/O operation,

1. The device driver loads the appropriate registers within the device controller.
2. The device controller, in turn, examines the contents of these registers to determine what action to take (such as "read a character from the keyboard").
3. The controller starts the transfer of data from the device to its local buffer.
4. Once the transfer of data is complete, the device controller informs the device driver via an interrupt that it has finished its operation.
5. The device driver then returns control to the operating system, possibly returning the data or a pointer to the data if the operation was a read.

- ✓ This form of interrupt-driven I/O is fine for moving small amounts of data but can produce high overhead when used for bulk data movement such as disk I/O.

- ✓ To solve this problem, **direct memory access (DMA)** is used. After setting up buffers, pointers, and counters for the I/O device, the device controller transfers an entire block of data directly to or from its own buffer storage to memory, with no intervention by the CPU.

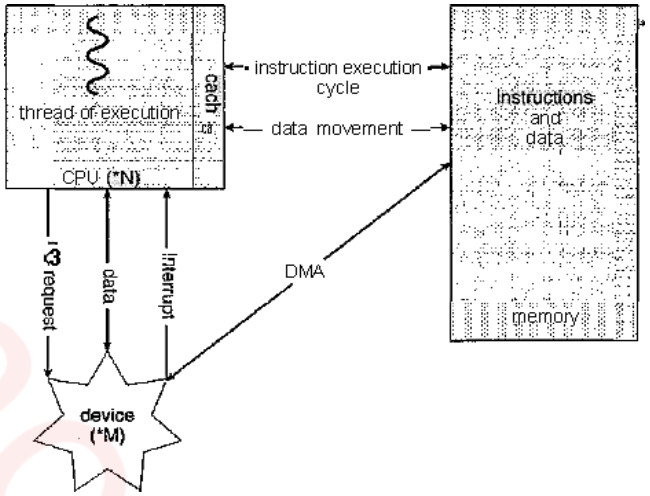


Figure 1.5 how a modern computer system works

- ✓ Figure 1.5 shows the interplay of all components of a computer system.

Computer-System Architecture

- ✓ Single-Processor Systems
- ✓ Multiprocessor Systems
- ✓ Clustered Systems

Single-Processor Systems

- ✓ On this system, there is one main CPU.
- ✓ The CPU is capable of executing a general-purpose instruction set, including instructions from user processes. Almost all systems have other special-purpose processors as well.
- ✓ The special purpose processor may come in the form of **device-specific processors**, such as disk, keyboard, and graphics controllers; or, on mainframes, they may come in the form of more general-purpose processors, such as I/O processors that move data rapidly among the components of the system.
- ✓ All of these **special-purpose processors run a limited instruction set and do not run user processes**.
- ✓ For example, a disk-controller microprocessor receives a sequence of requests from the main CPU and implements its own disk queue and scheduling algorithm.
- ✓ This arrangement relieves the main CPU of the overhead of disk scheduling. PCs contain a microprocessor in the keyboard to convert the keystrokes into codes to be sent to the CPU.

vtunotesforall.in

In other systems or circumstances, special-purpose processors are low-level components built in to the hardware.

Multiprocessor Systems

- ✓ **Multiprocessor systems** are having two or more processors in close communication, sharing the computer
    1. bus
    2. the clock,
    3. memory, and
    4. peripheral devices.

- ✓ **Multiprocessor systems have three main advantages:**

    1. Increased throughput. By increasing the number of processors, we expect o get more work done in less time.

    2. Economy of scale. Multiprocessor systems can cost less than equivalent multiple single-processor systems, because they can share peripherals, mass storage, and power supplies.

    3. Increased reliability. If functions can be distributed properly among several processors, then the failure of one processor will not halt the system, only slow it down. If we have ten processors and one fails, then each of the remaining nine processors can pick up a share of the work of the failed processor.

       The ability to continue providing service proportional to the level of surviving hardware is called ***graceful degradation.***

       Some systems go beyond graceful degradation and are called ***fault tolerant,*** because they can suffer a failure of any single component and still continue operation.

- ✓ The multiple-processor systems in use today are of two types.

    1. Some systems use **asymmetric multiprocessing,** in which each processor is assigned a specific task. A ***master processor controls the system***; the ***other processors either look to the master for instruction or have predefined tasks***. This ***scheme defines a master-slave relationship***. ***The master processor schedules and allocates work to the slave processors.***

    Ex: Sun's operating system SunOS Version 4 provided asymmetric multiprocessing,

    2. The other systems use **symmetric multiprocessing (SMP),** in which each processor performs all tasks within the operating system. SMP ***means that all processors are peers***; ***no master-slave relationship*** exists between processors. Figure 1.6 illustrates a typical SMP architecture.

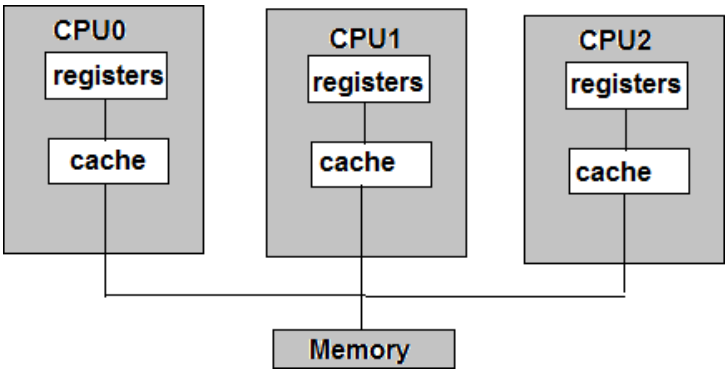


Figure 1.6 Symmetric multiprocessing architecture.

- ✓ A recent trend in CPU design is to include multiple compute **cores** on a single chip. In essence, these are multiprocessor chips. Two-way chips are becoming mainstream, while N-way chips are going to be common in high-end systems.
- ✓ In Figure 1.7, we have a dual-core design with two cores on the same chip. In this design, each core has its own register set as well as its own local cache; other designs might use a shared cache or a combination of local and shared caches.

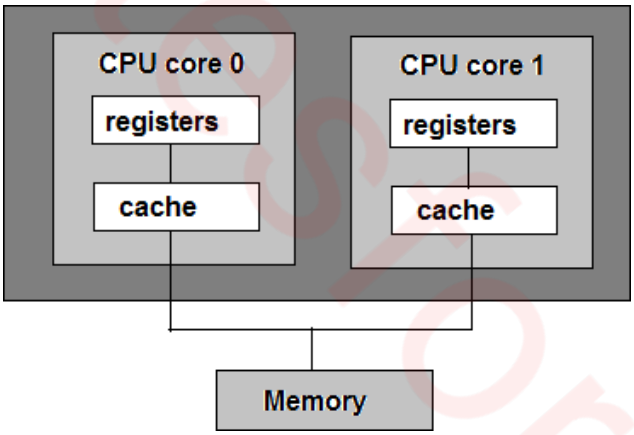


Figure 1.7 A dual-core design with two cores placed on the same chip

Clustered Systems

- ✓ **Clustered systems gather together multiple CPUs to accomplish computational work,** in this they are composed of two or more individual systems coupled together.
- ✓ They are linked via a **local-area network (LAN)** or a faster interconnects such as Infini Band.
- ✓ ***Advantage***: Clustering usually used to provide **high-availability** service; that is, service will continue even if one or more systems in the cluster fail. High availability is generally obtained by adding a level of redundancy in the system.

- ✓ **Working:**

1. A layer of cluster software runs on the cluster nodes.
2. Each node can monitor one or more of the others (over the LAN).
3. If the monitored machine fails, the monitoring machine can take ownership of its storage and restart the applications that were running on the failed machine.

- ✓ Clustering are of two types

1. asymmetric clustering
2. Symmetric clustering

- ✓ In **asymmetric clustering,** one machine is in **hot-standby mode** while the other is running the applications. The hot-standby host machine does nothing but monitor the active server. If that server fails, the hot-standby host becomes the active server.

- ✓ In **symmetric mode,** two or more hosts are running applications, and are monitoring each other. This mode is obviously more efficient, as it uses all of the available hardware. It does require that more than one application be available to run.

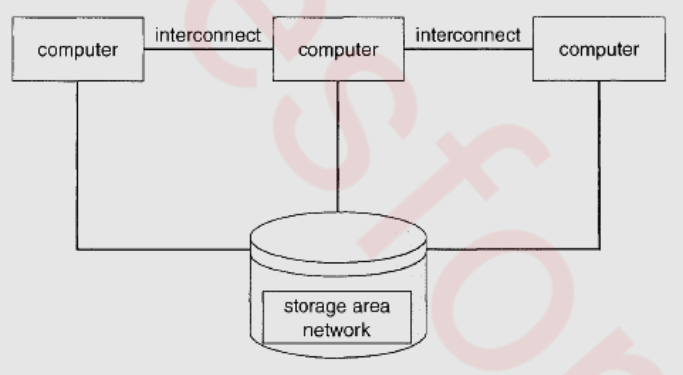


Figure 1.8 General structure of a clustered system

Operating System Structure

- ✓ A single user cannot, in general, keep either the CPU or the I/O devices busy at all times.

- ✓ **Multiprogramming** increases CPU utilization by organizing jobs (code and data) so that the CPU always has one to execute. The idea is as follows: The operating system keeps several jobs in memory simultaneously (Figure 1.9).

- ✓ This set of jobs can be a subset of the jobs kept in the job pool—which contains all jobs that enter the system—since the number of jobs that can be kept simultaneously in memory is usually smaller than the number of jobs that can be kept in the job pool.

- ✓ The operating system picks and begins to execute one of the jobs in memory. Eventually, the job may have to wait for some task, such as an I/O operation, to complete.

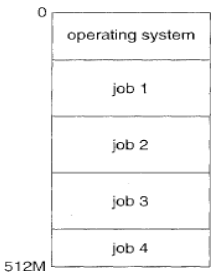


Figure 1.9 Memory layout for a multiprogrammed system

- ✓ In a non-multi-programmed system, the CPU would sit idle. In a multi-programmed system, the operating system simply switches to, and executes, another job. When *that* job needs to wait, the CPU is switched to *another* job, and so on.

- ✓ Eventually, the first job finishes waiting and gets the CPU back. As long as at least one job needs to execute, the CPU is never idle.

- ✓ Multi-programmed systems provide an environment in which the various system resources (for example, CPU, memory, and peripheral devices) are utilized effectively, but they do not provide for user interaction with the computer system.

- ✓ **Time sharing** (or **multitasking)** is a logical extension of multiprogramming. In time-sharing systems, the CPU executes multiple jobs by switching among them, but the switches occur so frequently that the users can interact with each program while it is running.

- ✓ Time sharing requires an **interactive** (or **hands-on) computer system,** which provides direct communication between the user and the system. The user gives instructions to the operating system or to a program directly, using a input device such as a keyboard or a mouse, and waits for immediate results on an output device.

- ✓ Accordingly, the **response time** should be short—typically less than one second. A time-shared operating system allows many users to share the computer simultaneously.

- ✓ A time-shared operating system uses **CPU scheduling** and multiprogramming to provide each user with a small portion of a time-shared computer. Each user has at least one separate program in memory.

- ✓ In addition, if several jobs are ready to run at the same time, the system must choose among them. Making this decision is **CPU scheduling**.

- ✓ Finally, running multiple jobs concurrently requires that their ability to affect one another be limited in all phases of the operating system, including process scheduling, disk storage, and memory management.

Operating-System Operations

1) Dual-Mode Operation

- ✓ In order to ensure the proper execution of the operating system, we must be able to distinguish between the execution ***of operating-system code*** and ***user defined code***.
- ✓ two separate **modes** of operation:
  user mode and
  kernel mode (also called **supervisor mode, system mode, or privileged mode).**
- ✓ A bit, called the **mode bit,** is added to the hardware of the computer to indicate the current mode: kernel (0) or user (1).
- ✓ ***When the computer system is executing on behalf of a user application, the system is in user mode.***
- ✓ ***When a user application requests a service from the operating system (via a system call), it must transition from user to kernel mode*** **to fulfill the request.** This is shown in Figure 1.10.

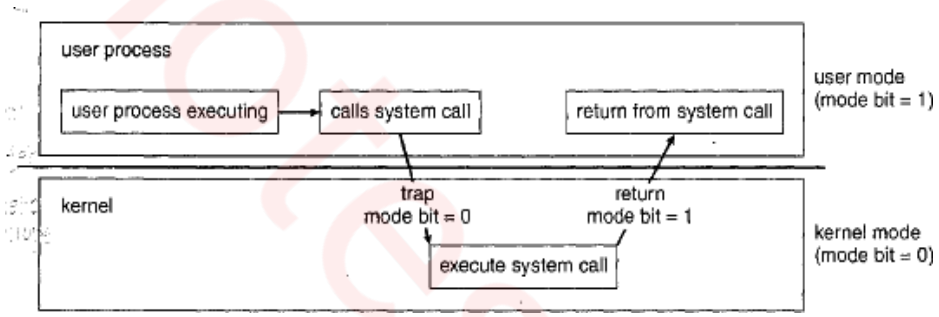


Figure 1.10 Transition from user to kernel mode

- ✓ This architectural enhancement is useful for many other aspects of system operation as well.

  1. At system boot time, the hardware starts in kernel mode.
  2. The operating system is then loaded and starts user applications in user mode.
  3. Whenever a trap or interrupt occurs, the hardware switches from user mode to kernel mode (i.e changes the state of the mode bit to 0).
  4. Thus, whenever the operating system gains control of the computer, it is in kernel mode.
  5. The system always switches to user mode (by setting the mode bit to 1) before passing control to a user program.

- ✓ The dual mode of operation provides us with the means for protecting the operating system from errant users—and errant users from one another.

- ✓ We accomplish this protection by designating some of the machine instructions that may cause harm as **privileged instructions.** The hardware allows privileged instructions to be executed only in kernel mode.

- ✓ If an attempt is made to execute a privileged instruction in user mode, the hardware does not execute the instruction but rather treats it as illegal and traps it to the operating system. The instruction to switch to user mode is an example of a privileged instruction.

- ✓ When control is given to a user application, the mode is set to user mode. Eventually, control is switched back to the operating system via an interrupt, a trap, or a system call.

2) Timer

- ✓ We must prevent a user program from getting stuck in an infinite loop or not calling system services and never returning control to the operating system.

- ✓ A **timer** can be set to interrupt the computer after a specified period.

- ✓ The period may be fixed (for example,1/60 second) or variable (for example, from 1 millisecond to 1 second).

- ✓ A **variable timer** is generally implemented by a fixed-rate clock and a counter. The operating system sets the counter. Every time the clock ticks, the counter is decremented. When the counter reaches 0, an interrupt occurs.

- ✓ Before turning over control to the user, the operating system ensures that the timer is set to interrupt.

- ✓ Thus, we can use the timer to prevent a user program from running too long. A simple technique is to initialize a counter with the amount of time that a program is allowed to run.

- ✓ As long as the counter is positive, control is returned to the user program. When the counter becomes negative, the operating system terminates the program for exceeding the assigned time limit.

Process Management

- ✓ A program in execution is a process.

- ✓ A process needs certain resources—including CPU time, memory, files, and I/O devices—to accomplish its task.

- ✓ When the process terminates, the operating system will reclaim any reusable resource

- ✓ A program by itself is not a process; **a program is a *passive* entity**, such as the contents of a file stored on disk, whereas **a process is an *active* entity**.

vtunotesforall.in

- ✓ A single-threaded process has one program counter specifying the next instruction to execute.
- ✓ A multithreaded process has multiple program counters, each pointing to the next instruction to execute.
- ✓ The operating system is responsible for the following activities in connection with process management:

1. Creating and deleting both user and system processes.
2. Suspending and resuming processes
3. Providing mechanisms for process synchronization
4. Providing mechanisms for process communication
5. Providing mechanisms for deadlock handling

Memory Management

- ✓ Main memory is a large array of words or bytes. Main memory is a repository of quickly accessible data shared by the CPU and I/O devices.
- ✓ The CPU ***reads instructions from main memory during the instruction-fetch cycle*** and both ***reads and writes data from main memory during the data-fetch cycle***.
- ✓ The main memory is generally the only large storage device that the CPU is able to address and access directly.
- ✓ To improve both the utilization of the CPU and the speed of the computer's response to its users, general-purpose computers must keep several programs in memory, creating a need for memory management.
- ✓ The operating system is responsible for the following activities in connection with memory management:

1. Keeping track of which parts of memory are currently being used and by whom
2. Deciding which processes (or parts thereof) and data to move into and out of memory
3. Allocating and de-allocating memory space as needed.

Storage Management

1. **File-System Management**
2. **Mass-Storage Management**
3. **Caching**

vtunotesforall.in

1. File-System Management

- ✓ A file is a collection of related information defined by its creator. Commonly files represent programs (both source and object forms) and data.
- ✓ Data files maybe numeric, alphabetic, alphanumeric, or binary.
- ✓ Files are normally organized into directories to make them easier to use.
- ✓ Finally, when multiple users have access to files, it may be desirable to control by whom and in what ways (for example, read, write, append) files may be accessed.
- ✓ The operating system is responsible for the following activities in connection with file management:
  1. Creating and deleting files
  2. Creating and deleting directories to organize files
  3. Supporting primitives for manipulating files and directories
  4. Mapping files onto secondary storage
  5. Backing up files on stable (nonvolatile) storage media

2. Mass-Storage Management

- ✓ Most programs are stored on a disk until loaded into memory and then use the disk as both the source and destination of their processing.
- ✓ The operating system is responsible for the following activities in connection with disk management:
  - Free-space management
  - Storage allocation
  - Disk scheduling.
- ✓ Because secondary storage is used frequently, it must be used efficiently. The entire speed of operation of a computer may hinge on the speeds of the disk subsystem and of the algorithms that manipulate that subsystem.
- ✓ There are, however, many uses for storage that is slower and lower in than secondary storage. Backups of disk data, seldom-used data, and long-term archival storage are some examples.
- ✓ Magnetic tape drives and their tapes and CD and DVD drives and platters are typical tertiary storage devices.

vtuinotesforall.in

3. Caching

- ✓ Caching is an important principle of computer systems.
- ✓ Information is normally kept in some storage system (such as main memory).
- ✓ As it is used, it is copied into a faster storage system—the cache—on a temporary basis.
- ✓ When we need a particular piece of information,
  1. We first check whether it is in the cache.
  2. If information is in cache, we use the information directly from the cache;
  3. If it is not there, we use the information from the source, putting a copy in the cache under the assumption that we will need it again soon.
- ✓ Without this cache, the CPU would have to wait several cycles while an instruction was fetched from main memory.
- ✓ Figure 1.11 shows a storage performance comparison in large workstations and small servers that shows the need for caching.

Level	1	2	3	4
Name	registers	cache	main memory	disk storage
Typical size	< 1 KB	< 16 MB	< 64 GB	> 100 GB
Implementation technology	custom memory with multiple ports, CMOS	on-chip or off-chip CMOS SRAM	CMOS DRAM	magnetic disk
Access time (ns)	0.25 – 0.5	0.5 – 25	80 – 250	5,000,000
Bandwidth (MB/sec)	20,000 – 100,000	5000 – 10,000	1000 – 5000	20 – 150
Managed by	compiler	hardware	operating system	operating system
Backed by	cache	main memory	disk	CD or tape

- ✓ Main memory can be viewed as a fast cache for secondary storage, since data in secondary storage must be copied into main memory for use, and data must be in main memory before being moved to secondary storage for safe keeping.
- ✓ The ***increment operation*** proceeds by
  1. first issuing an I/O operation to copy the disk block on which A resides to main memory.
  2. This operation is followed by copying A to the cache and to an internal register.
  3. Thus, the copy of A appears in several places: on the magnetic disk, in main memory, in the cache, and in an internal register (see Figure 1.12).
  4. Once the increment takes place in the internal register, the value of A differs in the various storage systems.

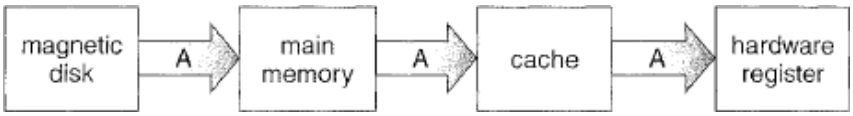


Figure 1.12 Migration of integer A from disk to register

4. I/O Systems

- ✓ The peculiarities of I/O devices are hidden from the bulk of the operating system itself by the I/O subsystem.
- ✓ The I/O subsystem consists of several components:
  1. A memory-management component that includes buffering, caching, and spooling
  2. A general device-driver interface
  3. Drivers for specific hardware devices.
- ✓ Only the device driver knows the peculiarities of the specific device to which it is assigned.

Protection and Security

- ✓ *Protection is a mechanism of controlling the access of processes or users to the resources defined by a computer system.*
- ✓ Protection can improve reliability by detecting latent errors at the interfaces between component subsystems.
- ✓ Security means defense of the system from internal and external attacks.
- ✓ The attacks include
  1. Viruses and worms
  2. Denial of Service (DOS)
  3. Identity theft
- ✓ Protection and security require the system to be able to distinguish among all its users. Most operating systems maintain a list of user names and associated user identifiers (user IDs).
- ✓ When a user logs in to the system, the authentication stage determines the appropriate user ID for the user. That user ID is associated with all of the user's processes and threads.

Distributed Systems

- ✓ A distributed system is a collection of physically separate, possibly heterogeneous computer systems that are networked to provide the users with access to the various resources that the system maintains.

vtunotesforall.in

- ✓ Access to a shared resource increases
  1. computation speed
  2. functionality
  3. data availability and
  4. reliability

- ✓ A network is a communication path between two or more systems.

- ✓ Networks vary by the
  1. protocols used
  2. distances between nodes and
  3. Transport media.

- ✓ Common network protocol are
  1. TCP/IP
  2. ATM.

- ✓ Networks are characterized based on the distances between their nodes.
  1. A local-area network (LAN) connects computers within a building.
  2. A wide-area network (WAN) usually links buildings, cities, or countries.
  3. A metropolitan-area network (MAN) could link buildings within a city.

- ✓ The media to carry networks are equally varied. They include
  1. copper wires,
  2. fiber strands, and
  3. wireless transmissions.

Special-Purpose Systems

1. Real-Time Embedded Systems
2. Multimedia Systems
3. Handheld Systems

1. **Real-Time Embedded Systems**

- ✓ Embedded system devices are found everywhere, from car engines and manufacturing robots to VCRs and microwave ovens.

- ✓ They tend to have very specific tasks.

- ✓ The systems they run on are usually primitive, and so the operating systems provide limited features.

- ✓ Usually, they prefer to spend their time monitoring & managing hardware devices such as

  1. Automobile engines and

  2. Robotic arms.

vtunotesforall.in

- ✓ Embedded systems almost always run real-time operating systems.
- ✓ A real-time system is used when rigid time requirements have been placed on the operation of a processor.
- ✓ A real-time system has **well-defined, fixed time constraints**. Processing *must be* done within the defined constraints, or the system will fail.

2. Multimedia Systems

- ✓ Multimedia data consist of audio and video files as well as conventional files.
- ✓ These data differ from conventional data in that multimedia data must be delivered(streamed) according to certain time restrictions.
- ✓ Multimedia describes a wide range of applications. These include

1. audio files such as MP3
2. DVD movies
3. video conferencing
4. live webcasts of speeches

3. Handheld Systems

- ✓ Handheld systems include
  1. PDAs and
  2. cellular telephones.
- ✓ Main challenge faced by developers of handheld systems: Limited size of devices.
- ✓ Because of small size, most handheld devices have a
  1. small amount of memory,
  2. slow processors, and
  3. small display screens.

Computing Environments

1. Traditional Computing
2. Client-Server Computing
3. Peer-to-Peer Computing
4. Web-Based Computing

1. Traditional Computing

- ✓ Used in office environment:
  - PCs connected to a network, with servers providing file and print services.
- ✓ Used in home networks:

vtunotesforall.in

- At home, most users had a single computer with a slow modem.
- Some homes have firewalls to protect their networks from security breaches.

✓ Web technologies are stretching the boundaries of traditional computing.

- Companies establish portals, which provide web accessibility to their internal servers.
- Network computers are terminals that understand web computing.
- Handheld PDAs can connect to wireless networks to use company's web portal.

✓ Systems were either batch or interactive.

- Batch system processed jobs in bulk, with predetermined input.
- Interactive systems waited for input from users.

2. Client-Server Computing

✓ Servers can be broadly categorized as (Figure 1.10):

1) Compute servers and
2) File servers

1. **Compute-server**
   - System provides an interface to which a client can send a request to perform an action (for example, read data).
   - In response, the server executes the action and sends back results to the client.
2. **File-server** system provides a file-system interface where clients can create, read, and delete files.
   For example: web server that delivers files to clients running web browsers

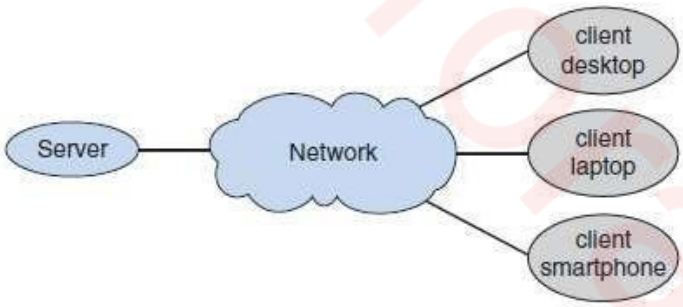


Figure 1.10 General structure of a client–server system

3. Peer-to-Peer Computing

✓ All nodes are considered peers, and each may act as either a client or a server(Figure 1.11).

✓ Advantage:

In a client-server system, the server is a bottleneck; but in a peer-to-peer system, services can be provided by several nodes distributed throughout the network.

- ✓ A node must first join the network of peers.
- ✓ Determining what services are available is done in one of two general ways:
  1. When a node joins a network, it registers its service with a centralized lookup service on the network.
     - Any node desiring a specific service first contacts this centralized lookup service to determine which node provides the service.
  2. A peer broadcasts a request for the service to all other nodes in the network. The node (or nodes) providing that service responds to the peer.

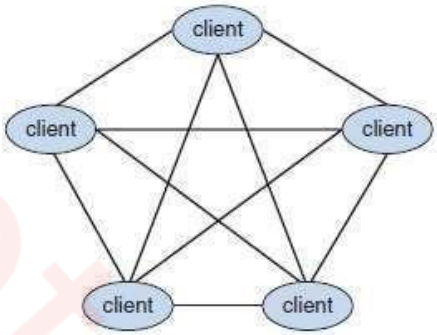


Figure 1.11 Peer-to-peer systems with no centralized service.

4. Web-Based Computing

- ✓ This includes
  1. PC
  2. handheld PDA &
  3. cell phones
- ✓ Load balancer is a new category of devices to manage web traffic among similar servers.
- ✓ In load balancing, network connection is distributed among a pool of similar servers.
- ✓ More devices becoming networked to allow web access
- ✓ Use of operating systems like Windows 95, client-side, have evolved into Linux and Windows XP, which can be clients and servers

vtunotesforall.in

OPERATING-SYSTEM STRUCTURES

Operating System Services

- ✓ An operating system provides an environment for the execution of programs.
- ✓ It provides certain services to programs and to the users of those programs.

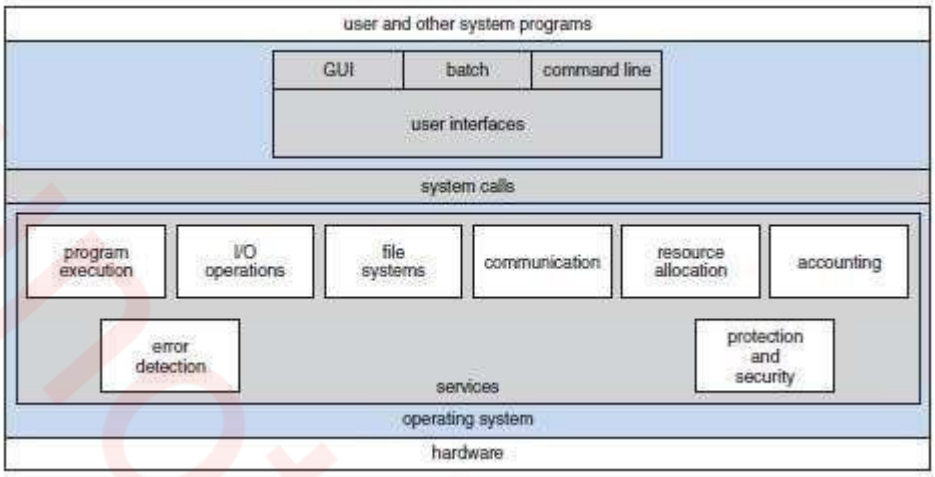


Figure 1.12 A view of OS services

- ✓ OS provides two sets of services,
  - i. one set of service for user
  - ii. and another set of services for system

Operating-system services provide functions that are helpful to the user

1. **User Interface**
   - Almost all OS have a user-interface (UI).
   - Different interfaces are:

     i) CLI (Command Line Interface)

     This uses
     - text commands and
     - method for entering the text commands.

     ii) Batch Interface
     - Commands & directives to control those commands are entered into files, and those files are executed.

     iii) GUI (Graphical User Interface)
     - The interface is a window-system with a pointing-device to
       1. direct I/0
       2. choose from menus and
       3. Make selections.
2. **Program execution**. *The system must be able to load a program into memory and to run that program***.** The program must be able to end its execution, either normally or abnormally.
3. **I/O operations.** *A running program may require I/O, which may involve a file or an I/O device.* For specific devices, special functions may be desired.

4. **File-system manipulation**. Programs need to
    1. read and
    2. write files and directories.
    3. to create and delete file them by name,
    4. search for a given file, and list file information.
    5. allow or deny access to files or directories based on file ownership.
5. **Communications**
    - ✓ In some situations, one process needs to communicate with another process.
    - ✓ Communications may be implemented via
        1. Shared memory or
        2. Message passing
    - ✓ In message passing, packets of information are moved between processes by OS.
6. **Error detection**
    - ✓ Errors may occur in
        - CPU & memory-hardware (ex: power failure)
        - I/O devices (ex: lack of paper in the printer) and
        - user program (ex: arithmetic overflow
    - ✓ For each type of error, OS should take appropriate action to ensure correct & consistent computing.

Another set of operating-system functions

1) Resource Allocation

- When multiple users are logged on the system at the same time, resources must be allocated to each of them.
- The OS manages different types of resources.
- Some resources (say CPU cycles) may have special allocation code.
- Other resources (say I/O devices) may have general request & release code.

2) Accounting

- We want to keep track of
    i. which users use how many resources and
    ii. which kinds of resources.
- This record keeping may be used for
    i. accounting (so that users can be billed) or
    ii. gathering usage-statistics.

3) Protection

- When several separate processes execute concurrently, it should not be possible for one process to interfere with the others or with the OS itself.
- Protection involves ensuring that all access to resources is controlled.
- Security starts with each user having authenticated to the system by means of a password.

User Operating-System Interface

- ✓ There are two fundamental approaches for users to interface with the operating system. One technique is to provide a command-line interface or command interpreter

that allows users to directly enter commands that are to be performed by the operating system. The second approach allows the user to interface with the operating system via a graphical user interface or GUI.

System Calls

- ✓ System calls provide an interface to the services made available by an operating system.
- ✓ These calls are generally available as routines written in C and C++
- ✓ The programmers design programs according to an API.
- ✓ The API
  → defines a set of functions that are available to the programmer (Figure 1.15).
  → includes the parameters passed to functions and the return values.
- ✓ The functions that make up an API invoke the actual system-calls on behalf of the programmer.
- ✓ Benefits of API:
  1) Program portability.
  2) Actual system-calls are more detailed (and difficult) to work with than the API available to the programmer.
- ✓ Three general methods are used to pass parameters to the OS:
  1) via registers.
  2) Using a table in memory & the address is passed as a parameter in a register (Figure 1.16).
  3) The use of a stack is also possible where parameters are pushed onto a stack and popped off the stack by the OS.

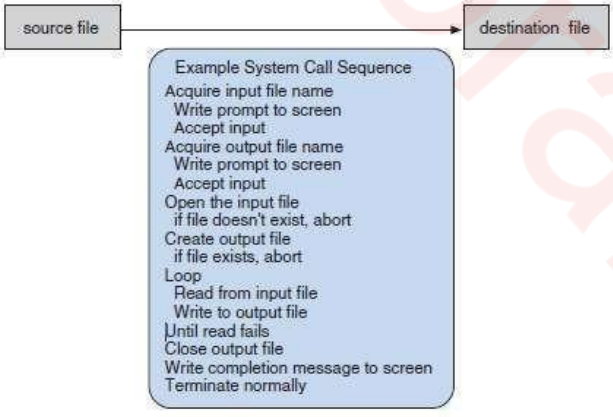


Figure 1.15 Example of how system calls are used

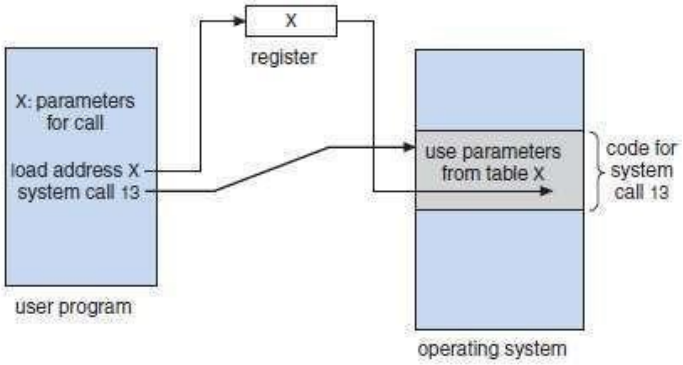


Figure 1.16 Passing of parameters as a table

Types of System Calls

2) Process control
3) File management
4) Device management
5) Information maintenance
6) Communications

1) Process Control

✓ System calls used:
- end, abort
- load, execute
- create process, terminate process
- get process attributes, set process attributes
- wait for time
- wait event, signal event
- allocate and free memory

✓ A running program needs to be able to halt its execution either normally (**end**) or abnormally (**abort**).

✓ If program runs into a problem, error message may be generated and dumped into a file. This file can be examined by a debugger to determine the cause of the problem.

✓ The OS must transfer control to the next invoking command interpreter.
- Command interpreter then reads next command.
- In interactive system, the command interpreter simply continues with next command.
- In GUI system, a pop-up window will request action from user. How to deal with new process?

✓ A process executing one program can **load** and **execute** another program.

✓ Where to return control when the loaded program terminates?

vturesponse.in

The answer depends on the existing program:

1) If control returns to the existing program when the new program terminates, we must save the memory image of the existing program. (Thus, we have effectively created a mechanism for one program to call another program).
2) If both programs continue concurrently, we **create**d a new process to be multiprogrammed.

✓ We should be able to control the execution of a process. i.e. we should be able to determine and reset the attributes of a process such as:
- Job's priority or
- Maximum execution time

✓ We may also want to **terminate** process that we created if we find that it
- is incorrect or
- is no longer needed.

✓ We may need to **wait** for processes to finish their execution. We may want to wait for a specific event to occur.

✓ The processes should then signal when that event has occurred.

2) File Management

✓ System calls used:
- create file, delete file
- open, close
- read, write, reposition
- get file attributes, set file attributes

✓ Working procedure:
1) We need to **create** and **delete** files.
2) Once the file is created,
   - we need to open it and to use it.
   - we may also read or write.
3) Finally, we need to **close** the file.

✓ We need to be able to
- determine the values of file-attributes and
- reset the file-attributes if necessary.

✓ File attributes include
- file name
- file type
- protection codes and
- Accounting information.

vtunotesforall.in

3) Device Management

✓ System calls used:
- request device, release device;
- read, write, reposition;
- get device attributes, set device attributes;
- logically attach or detach devices.

✓ A program may need additional resources to execute.

✓ Additional resources may be
→ memory
→ tape drives or
→ files.

✓ If the resources are available, they can be granted, and control can be returned to the user program; if the resources are unavailable, the program may have to wait until sufficient resources are available.

✓ Files can be thought of as virtual devices. Thus, many of the system calls used for files are also used for devices.

✓ In multi-user environment,
1) We must first **request** the device, to ensure exclusive use of it.
2) After we are finished with the device, we must **release** it.

✓ Once the device has been requested (and allocated), we can read and write the device.

✓ Due to lot of similarity between I/O devices and files, OS (like UNIX) merges the two into a combined file-device structure**.**

✓ UNIX merges I/O devices and files into a combined file-device structure**.**

4) Information Maintenance

✓ System calls used:
- get time or date, set time or date
- get system data, set system data
- get process, file, or device attributes
- set process, file, or device attributes

✓ Many system calls exist simply for the purpose of transferring information between the user program and the OS.
For ex,
1) Most systems have a system call to return
   - current time and
   - current date.
2) Other system calls may return information about the system, such as
   → number of current users

vtunotesforall.in

→ version number of the OS
→ amount of free memory or disk space.

✓ The OS keeps information about all its processes, and there are system calls to access this information.

Communication

✓ System calls used:
- create, delete communication connection
- send, receive messages
- transfer status information
- attach or detach remote devices

✓ Two models of communication.
1) Message-passing model and
2) Shared Memory Model

Message Passing Model

✓ Information is exchanged through an IPC provided by OS. (IPC=inter process communication).

✓ Steps for communication:

1) Firstly, a connection must be opened using **open connection** system-call.
2) Each computer has a host-name, such as an IP name.
   Similarly, each process has a process-name, which is translated into an equivalent identifier. The **get hostid** & **get processid** system-calls do this translation.
3) Then, identifiers are passed to the **open** and **close** system-calls.
4) The recipient-process must give its permission for communication to take place with an **accept connection** system-call.(The processes that will be receiving connections are called daemons processes).
5) Daemon processes
   - execute a **wait for connection** system-call and
   - are awakened when a connection is made.

6) Then, client & server exchange messages by **read message** and **write message** system calls.
7) Finally, the **close connection** system-call terminates the communication.

✓ Advantages:
1) Useful when smaller numbers of data need to be exchanged.
2) It is also easier to implement than is shared memory.

Shared Memory Model

- ✓ Processes use map memory system-calls to gain access to regions of memory owned by other processes.
- ✓ Several processes exchange information by reading and writing data in the shared memory.
- ✓ The shared memory
  - is determined by the processes and
  - Are not under the control of OS.
- ✓ The processes are also responsible for ensuring that they are not writing to the same location simultaneously.
- ✓ Advantage:
  1) Shared memory allows maximum speed and convenience of communication,
- ✓ Disadvantage:
  1) Problems exist in the areas of protection and synchronization.

System Programs

- ✓ They provide a convenient environment for program development and execution. (System programs also known as system utilities).
- ✓ They can be divided into these categories:
- ✓ Six categories of system-programs:
  - **File management**. These programs create, delete, copy, rename, print, dump, list, and generally manipulate files and directories.
  - **Status information**. Some programs simply ask the system for the date, time, amount of available memory or disk space, number of users, or similar status information. Others are more complex, providing detailed performance, logging, and debugging information
  - **File modification.** Several text editors may be available to create and modify the content of files stored on disk or other storage devices. There may also be special commands to search contents of files or perform transformations of the text.
  - **Programming-language support. Compilers, assemblers, debuggers and interpreters** for common programming languages (such as C, C++, Java, Visual Basic, and PERL) are often provided to the user with the operating system.

vtunotesforall.in

- **Program loading and execution.** Once a program is assembled or compiled, it must be loaded into memory to be executed. The system may provide absolute loaders, re-locatable loaders, linkage editors, and overlay loaders. Debugging systems for either higher-level languages or machine language are needed as well.

- **Communications.** These programs provide the mechanism for creating virtual connections among processes, users, and computer systems. They allow users to send messages to one another's screens, to browse web pages, to send electronic-mail messages, to log in remotely, or to transfer files from one machine to another

Operating system Design and Implementation

Design goals

✓ The first problem in designing a system is to
- Define goals and
- Define specifications.

✓ The design of the system will be affected by
- choice of hardware and
- type of system such as
  1) batch or time shared
  2) single user or multiuser

✓ Two basic groups of requirements: 1) User goals and
2) System goals

User goals

✓ The system should be
- convenient to use
- easy to learn and to use
- Reliable, safe, and fast.

System goals

✓ The system should be
- easy to design
- implement, and maintain
- Flexible, reliable, error free, and efficient.

Mechanisms and policies

✓ Mechanisms determine how to do something.

✓ Policies determine what will be done.

✓ Separating policy and mechanism is important for flexibility.

vtuprofessorall.in

✓ Policies change over time; mechanisms should be general.

Implementation

✓ OS's are nowadays written in higher-level languages like C/C++

✓ Advantages of higher-level languages:
- Faster development and
- OS is easier to port.

✓ Disadvantages of higher-level languages:
- Reduced speed and
- Increased storage requirements.

Operating-System Structure

Simple Structure

1. MS-DOS

✓ **Advantages:** These operating systems started as small, simple, and limited systems and then grew beyond their original scope. MS-DOS is an example of such a system.

✓ It was written to provide the most functionality in the least space, so it was not divided into modules carefully.

✓ Figure below shows its structure. In MS-DOS, the interfaces and levels of functionality are not well separated. For instance, application programs are able to access the basic I/O routines to write directly to the display and disk drives.

✓ **Disadvantages:** MS-DOS vulnerable to errant (or malicious) programs, causing entire system crashes when user programs fail.

✓ MS-DOS was also limited by the hardware of its era. Because the Intel 8088 for which it was written provides no dual mode and no hardware protection.

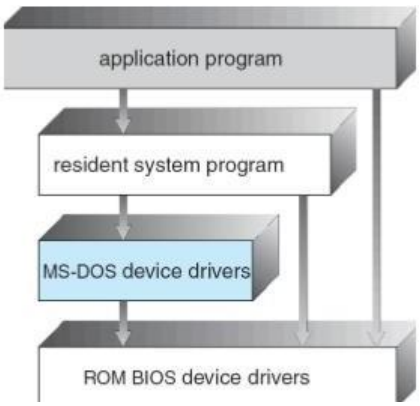


Figure MS-DOS layer structure

Vtunotesforall.in

2. UNIX

✓ It was initially limited by hardware functionality.

✓ Two parts of UNIX (Figure below):
1) Kernel and
2) System programs.

✓ The kernel is further separated into a series of interfaces and device drivers.

✓ Everything below the system-call interface and above the physical hardware is the kernel.

✓ The kernel provides following functions through system calls:
1. file system
2. CPU scheduling and
3. Memory management.

✓ Disadvantage:
1) Difficult to enhance, as changes in one section badly affects other areas.

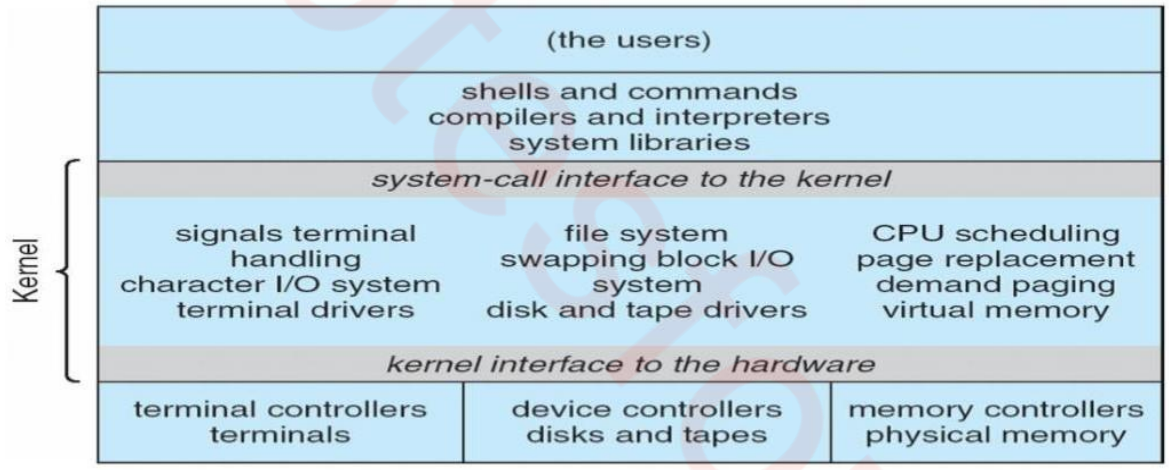


Figure UNIX system structure

Layered Approach

✓ The OS is divided into a number of layers.

✓ Each layer is built on the top of another layer.

✓ The bottom layer is the hardware.

The highest is the user interface (Figure below).

✓ A layer is an implementation of an abstract-object.

i.e. The object is made up of
- data and

- operations that can manipulate the data.

✓ The layer consists of a set of routines that can be invoked by higher-layers.

✓ Higher-layer
- does not need to know how lower-layer operations are implemented
- Needs to know only what lower-layer operations do.

✓ Advantage:
1) The layered approach is simplicity of construction and debugging.
2) Simplifies debugging and system verification.
3) The **design and implementation** of the system is simplified. Each layer is implemented with only those operations provided by lower level layers.
4) Each layer hides the existence of certain data structures, operations, and hardware from higher-level layers.

✓ Disadvantages:
1) Difficulty with the layered approach involves **appropriately defining the various layers**. Because a layer can use only lower-level layers, careful planning is necessary.
2) Less efficient than other types.

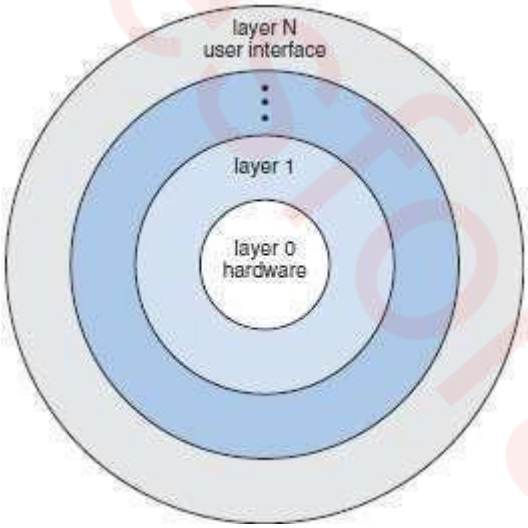


Figure : A layered operating system.

Microkernels

✓ Main function:
- To provide a communication facility between
  - client program and
  - Various services running in user-space.

✓ Communication is provided by message passing (Figure).

✓ All non-essential components are
- removed from the kernel and
- Implemented as system- & user-programs.

✓ Advantages:

1) Ease of extending the OS. (New services are added to user space w/o modification of kernel).
2) Easier to port from one hardware design to another.
3) Provides more security & reliability. (If a service fails, rest of the OS remains untouched.).
4) Provides minimal process and memory management.

✓ Disadvantage:

1) Performance decreases due to increased system function overhead.

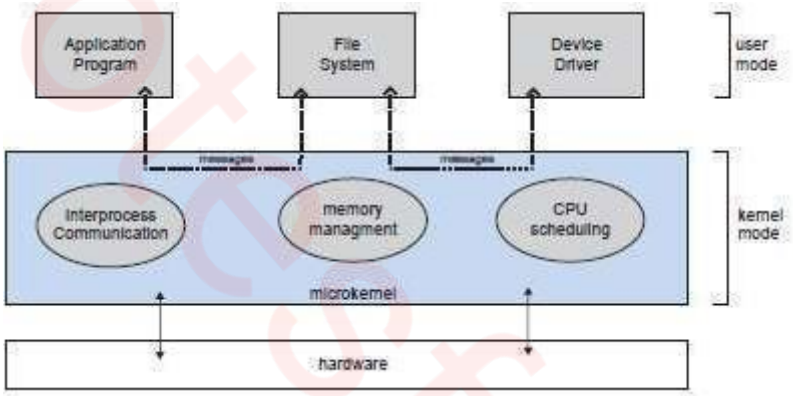


Figure Architecture of a typical microkernel

Modules

✓ The kernel has
- set of core components and
- Dynamic links in additional services during boot time(or run time).

✓ Seven types of modules in the kernel (Figure 1.21):
1) Scheduling classes
2) File systems
3) Loadable system calls
4) Executable formats
5) STREAMS modules
6) Miscellaneous
7) Device and bus drivers

✓ The top layers include
- application environments and
- Set of services providing a graphical interface to applications.

- ✓ Kernel environment consists primarily of
  - Mach microkernel and
  - BSD kernel.

- ✓ Mach provides
  - memory management;
  - support for RPCs & IPC and
  - Thread scheduling.

- ✓ BSD component provides
  - BSD command line interface
  - support for networking and file systems and
  - implementation of POSIX APIs

- ✓ The kernel environment provides an I/O kit for development of
  - device drivers and
  - Dynamic loadable modules (which Mac OS X refers to as kernel extensions).

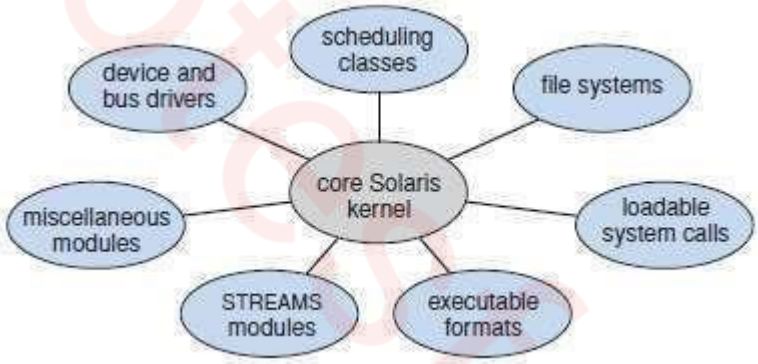


Figure 1.21 Solaris loadable modules

Virtual Machines

- ✓ The fundamental idea behind a virtual machine is to abstract the hardware of a single computer (the CPU, memory, disk drives, network interface cards, and so forth) into several different execution environments, thereby **creating the illusion** that each separate execution environment is running its own private computer.

- ✓ An operating system can create the illusion that a process has its own processor with its own (virtual) memory.

- ✓ Normally, a process has additional features, such as system calls and a file system that are not provided by the bare hardware

- ✓ The virtual-machine provides
  - an interface that is identical to the underlying hardware (Figure below).
  - a (virtual) copy of the underlying computer to each process.

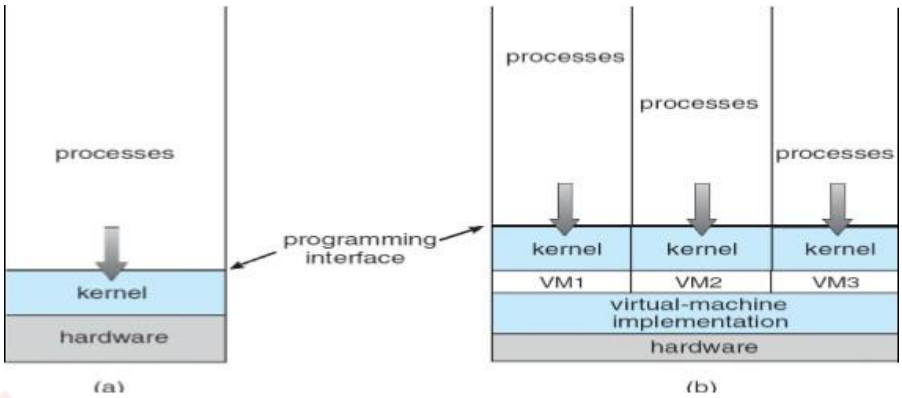


Figure: (a) Non-virtual machine, (b) Virtual machine

Benefits of virtual machine

1. Virtual machine is fundamentally related to being **able to share the same hardware** but runs different operating systems concurrently.

2. The **host system is protected** from the virtual machines. A virus inside a guest operating system might damage that operating system but is unlikely to affect the host or the other guests.

3. There is **no direct sharing of resources**. Two approaches to provide sharing have been implemented. First, it is possible to share a ***file-system volume*** and thus to share files. Second, it is possible to ***define a network of virtual machines***, each of which can send information over the virtual communications network.

4. A virtual-machine system is a **perfect vehicle** for operating-systems **research** and **development**. Changing an operating system is a difficult task.

5. Another advantage of virtual machines for developers is that **multiple operating systems** can be running on the developer's **workstation concurrently**.

6. Similarly, **quality-assurance engineers** can **test** their applications in multiple environments without buying, powering, and maintaining a computer for each environment.

7. A major advantage of virtual machines in **production data-center** use is system which involves taking two or more separate systems and running them in virtual machines on one system. Such physical-to-virtual conversions result in **resource optimization**, as many lightly used systems can be combined to create one more heavily used system.

8. Any **virtual machine will run on any virtualization platform**.

I. Para-virtualization

- ✓ Para-virtualization presents the guest with a system that is similar but not identical to the guest's preferred system.

- ✓ The guest must be modified to run on the Para-virtualized hardware. The gain for this extra work is more efficient use of resources and a smaller virtualization layer.
- ✓ The operating system and its devices are virtualized, providing processes within a container with the impression that they are the only processes on the system.
- ✓ One or more containers can be created, and each can have its own applications, network stacks, network address and ports, user accounts, and so on. CPU resources can be divided up among the containers and the system wide processes.

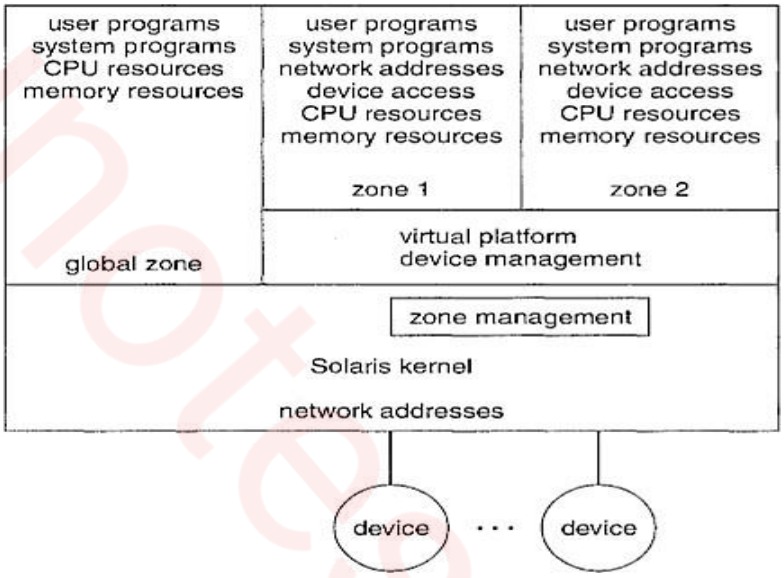


Figure: Solaris 10 with two containers

Examples

VMware

- ✓ VMware Workstation is a popular commercial application that abstracts Intel X86 and compatible hardware into isolated virtual machines.
- ✓ The architecture of such a system is shown in Figure below. In this scenario, Linux is running as the host operating system; and FreeBSD, Windows NT, and Windows XP are running as guest operating systems.
- ✓ The virtualization layer is the heart of VMware, as it abstracts the physical hardware into isolated virtual machines running as guest operating systems. Each virtual machine has its own virtual CPU, memory, disk drives, network interfaces, and so forth.

vtunotesforall.in

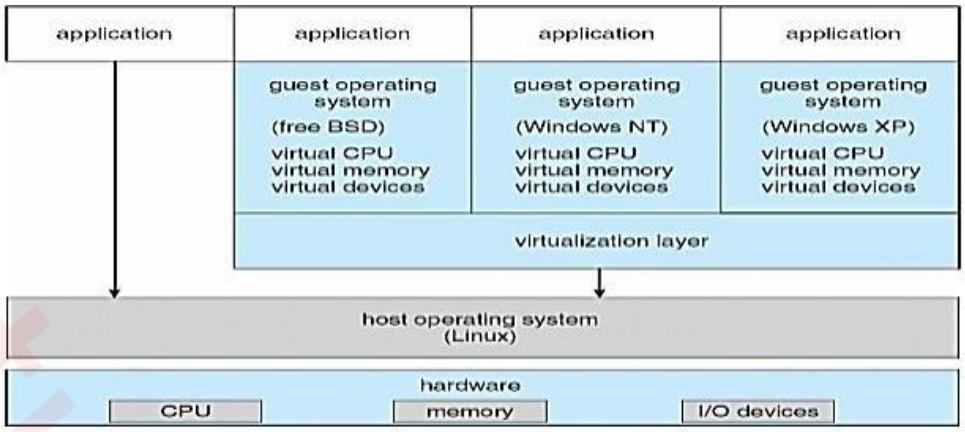


Figure: VMware architecture.

Operating-System Generation

- ✓ OS is designed to run on any of a class of machines.

  However, the system must be configured for each specific computer site

- ✓ SYSGEN is used for configuring a system for each specific computer site
- ✓ SYSGEN program must determine:

  1) What CPU will be used?
  2) How will boot disk be formatted?
  3) How much memory is available?
  4) What devices are available?
  5) What OS options are desired?

- ✓ A system-administrator can use the above information to modify a copy of the source code of the OS

System Boot

- ✓ Booting means starting a computer by loading the kernel.
- ✓ Bootstrap program is a code stored in ROM.
- ✓ The bootstrap program

  - locates the kernel
  - loads the kernel into main memory and
  - Starts execution of kernel.

- ✓ The bootstrap program can perform a variety of tasks. Usually, one task is to run diagnostics to determine the state of the machine
- ✓ OS must be made available to hardware so hardware can start it.

vtunotesforall.in